auto_ml 사용 설명서

이 문서는 두 파트로 구성됩니다.

- **Part I. 튜토리얼 (§1–5)** — 처음 사용하는 분이 설치부터 첫 실행까지 따라할 수 있는 단계별 안내.
- **Part II. 레퍼런스 (§6–20)** — 설정 파일의 모든 섹션을 모듈별로 정리. 각 섹션은 **역할 → 옵션 표 → YAML 예시 → 동작 메모** 4단으로 반복되어, 필요한 부분만 찾아 읽도록 설계되었습니다.

> **최근 변경 — 업그레이드 전에 꼭 확인하세요**: 실행 이력·롤백, 배포 안전장치(변수 누락 거부· 동시 실행 잠금), 정확도·안정성 버그 수정이 추가되었습니다. 운영 스크립트에 영향을 주는 동작 변화는 부록 F 에 정리했습니다.
>
> **운영 팟(컨테이너) 에 배포한다면**: CPU 할당에 맞춰 `training.n_jobs` · `scoring.n_jobs` 를 직접 적어야 합니다. 비워 두면 노드 전체 코어만큼 스레드를 만들어 수십 배 느려질 수 있습니다. → §20 운영 환경별 CPU 설정
>
> **실행 방식**: 패키지 설치 없이 `train/` · `inference/` 의 `.py` 스크립트로 실행합니다. → §2 전체 흐름, §5 빠른 시작

목차

Part I. 튜토리얼

1. 라이브러리 소개
2. 전체 흐름
3. 사전 준비
4. 설치
5. 빠른 시작
   - 5.1 실행 샘플 (명령 한 줄)
   - 5.2 단계별 직접 실행
   - 5.3 단계 재실행과 중간 산출물

Part II. 설정 레퍼런스

1. 설정 파일 구조와 YAML 문법
2. 데이터 입력 (top-level)
3. features.csv — 사용할 컬럼 정의
4. 전처리 `preprocessing`
5. 변수 선택 `feature_selection`

6. 학습 `training`
    - ↳ 최종 모델 학습 전략 — iteration_capping / cv_bagging
7. 튜닝 `tuning`
8. 모델 `models`
9. 앙상블 `ensemble`
10. 리포트 `reporting`
11. 스코어링 `scoring`
12. SHAP 해석 `explain`
13. 로깅 `logging`
14. 실행 이력·배포·롤백 `versioning`
15. 운영 환경별 CPU 설정 (팟·컨테이너)

부록

- A. 실행 명령어 모음
- B. 산출물 안내
- C. 트러블슈팅 FAQ
- D. 폐쇄망 이관
- E. 용어 사전
- F. 변경 이력과 업그레이드 안내

Part I. 튜토리얼

1. 라이브러리 소개

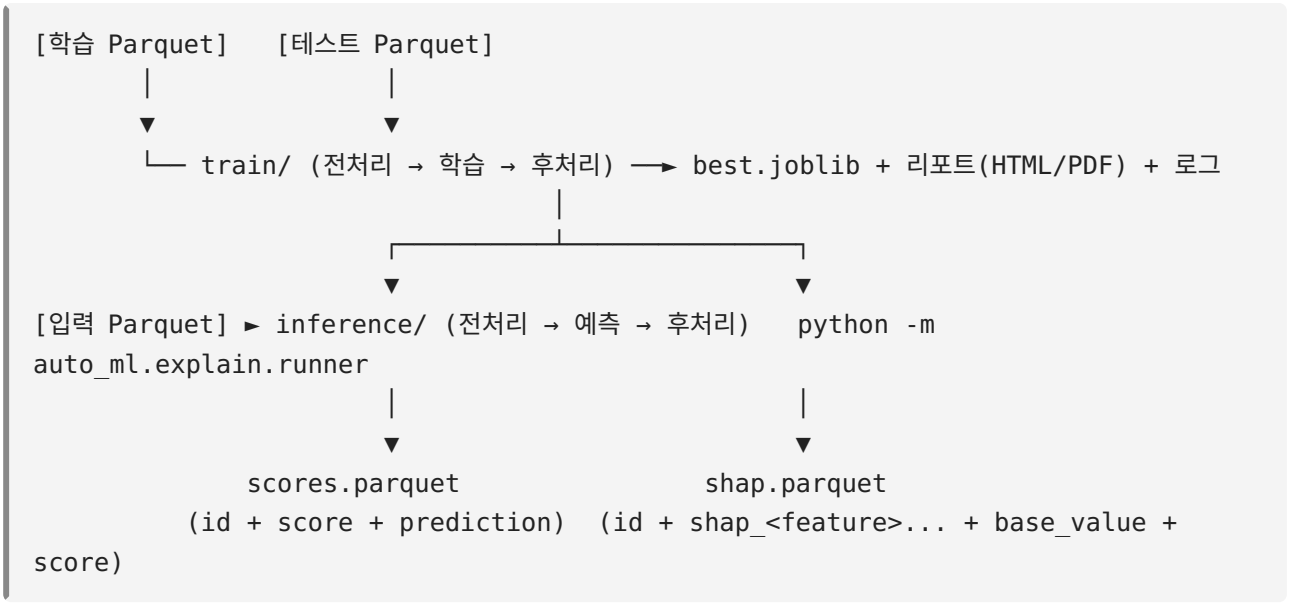
`auto_ml` 은 **이진 분류(0/1)** 문제를 자동으로 학습·스코어링하는 사내 라이브러리입니다. 이탈/비이탈, 사기/정상, 구매/비구매 같은 "예/아니오" 답을 맞히는 모든 문제에 쓸 수 있습니다.

ML 코드를 직접 작성할 필요는 없습니다. **YAML 설정 한 개**와 **컬럼 정의 CSV 한 개**만 준비하면 라이브러리가 다음을 수행합니다.

- 결측치 채우기, 이상치 윈저라이징, skew 변환, 스케일링
- (선택) Stability Selection 으로 변수 자동 선별
- LightGBM / XGBoost / CatBoost / ElasticNet 네 가지 모델 학습 (ElasticNet 은 기본 꺼짐)
- Optuna 로 하이퍼파라미터 베이지안 탐색
- 모델들을 가중 평균 앙상블로 결합 (설정에 따라 — § 14 참고)
- 테스트 점수 기준 best 모델 선정
- HTML / PDF 리포트 생성
- 학습 결과 한 덩어리(`best.joblib`)로 저장

학습이 끝나면 **`python inference/run_all.py`** 한 줄로 새 데이터에 점수를 매길 수 있습니다(cron 등 배치 운영용). 패키지 설치 없이 저장소의 `.py` 파일을 직접 실행합니다.

2. 전체 흐름



학습·추론은 각각 3단계 스크립트로 나뉩니다. `run_all.py` 는 3단계를 한 번에 실행합니다.

스크립트	단계	하는 일
`train/preprocess.py`	학습 1/3	데이터 로드·검증, 전처리 fit/transform, (선택) 변수 선택
`train/train.py`	학습 2/3	모델 튜닝 → CV → 최종 fit, best 선정
`train/postprocess.py`	학습 3/3	리포트, `best.joblib`·후보 artifact 저장, test 예측 export
`train/run_all.py`	학습 전체	위 1~3 일괄 실행 (보통 주 1회)
`inference/preprocess.py`	추론 1/3	`best.joblib` 의 전처리기로 입력 변환
`inference/predict.py`	추론 2/3	best 모델로 score(양성 확률) 계산
`inference/postprocess.py`	추론 3/3	임계값 적용, id 결합, `scores.parquet` 저장
`inference/run_all.py`	추론 전체	위 1~3 일괄 실행 (반복)

보조 명령어 (저장소 루트에서 실행): - `python -m auto_ml.explain.runner --config <yaml>` — 건별 변수 기여도(`shap.parquet`). - `python -m auto_ml.cli.set_best --config <yaml> --model <name>` — 학습 후 best 모델을 다른 후보로 교체(재학습 불필요).

단계 사이 산출물은 `<artifact_dir 의 부모>/work/{train,inference}/*.pkl.gz` 에 저장되며 `--work-dir` 로 위치를 바꿀 수 있습니다. 앞 단계 산출물이 없으면 다음 단계는 `FileNotFoundError` 로 멈춥니다. 자세한 내용은 § 5.3 참고.

설치 없이 바로 돌려보려면 § 5.1 실행 샘플 의 `python examples/sample/run_sample.py` 한 줄이면 됩니다.

---

3. 사전 준비

필요한 것

- **Python 3.10 이상** (`python --version` 으로 확인. 검증된 버전: 3.11, 3.13)
- **pip** (보통 Python 과 함께 설치)
- **Git** (소스 코드 clone 용)
- (SHAP 해석을 쓰려면) **shap** — `requirements.txt` 에 포함되어 있어 `pip install -r requirements.txt` 로 자동 설치됩니다. ElasticNet 모델의 SHAP 산출 시 필요합니다.
- (PDF 리포트를 만들려면) **WeasyPrint 시스템 의존성**
- macOS: `brew install pango libffi`
- Ubuntu/Debian: `sudo apt-get install libpango-1.0-0 libpangoft2-1.0-0`
- 미설치 시 학습/스코어링은 정상 동작하고 HTML 리포트만 생성됩니다.

디스크 / 메모리

- 더미 예제 기준 디스크 **500MB** 면 충분.
- 학습 데이터가 수백만 행이면 메모리 **8GB 이상** 권장.

---

4. 설치

터미널에서 한 줄씩 차례대로 실행합니다.

```
# 1) 프로젝트 폴더로 이동
cd auto-ml-briefly

# 2) 가상환경 생성 (시스템 Python 을 더럽히지 않도록)
python -m venv .venv

# 3) 가상환경 활성화
source .venv/bin/activate          # macOS / Linux
.venv\Scripts\Activate.ps1         # Windows (PowerShell)

# 4) 의존성 설치
pip install -r requirements.txt
```

`auto_ml` 패키지 자체는 설치하지 않습니다. 스크립트가 저장소 루트를 import 경로에 추가하므로, 의존 라이브러리만 설치하면 바로 실행할 수 있습니다.

설치를 확인합니다 (저장소 루트에서).

```
python train/run_all.py --help
python inference/run_all.py --help
```

도움말이 나오면 성공입니다. `ModuleNotFoundError` (예: `No module named 'lightgbm'`) 가 보이면 가상환경 활성화와 `pip install -r requirements.txt` 실행 여부를 다시 확인하세요.

---

5. 빠른 시작

진짜 데이터를 준비하기 전에, 샘플 데이터로 동작 여부를 한 번 확인합니다.

5.1 실행 샘플 (명령 한 줄)

`examples/sample/` 에 바로 돌려볼 수 있는 샘플이 있습니다. 합성 데이터(고객 2,800명)를 만든 뒤 학습 3단계 → 추론 3단계 스크립트를 순서대로 호출합니다. 규모를 작게 잡아 수십 초~1분 정도면 끝납니다.

```
python examples/sample/run_sample.py
```

파일	내용
`examples/sample/run_sample.py`	데이터 생성 + 6개 단계 스크립트 순차 실행 + 결과 요약 출력
`examples/sample/config.yaml`	샘플용 설정 (튜닝 5 trials, LGBM·XGB·ElasticNet + 앙상블, PDF 끔)
`examples/sample/features.csv`	사용할 컬럼 정의 (`memo_code` 는 `used=false` 예시)

샘플 데이터 구성:

컬럼	종류	비고
`num_0` ~ `num_5`	수치형	`num_0` 에 결측 약 3%
`amount`	수치형	오른쪽 꼬리가 긴 분포 → skew 변환 대상
`region` , `channel`	범주형	`channel` 에 결측 약 5%
`memo_code`	범주형	`features.csv` 에서 제외
`customer_id`	식별자	결과 파일에 보존
`target`	0/1	양성 비율 약 25%

실행이 끝나면 주요 산출물 경로와 `scores.parquet` 상위 5행이 출력됩니다. 모든 산출물은 `artifacts/sample/` 아래에 생깁니다.

옵션: - `--skip-data` — 이미 만든 `examples/sample/data/*.parquet` 를 그대로 사용. - `--only train` / `--only inference` — 학습 또는 추론만 실행 (추론만 실행하려면 학습 산출물이 있어야 합니다).

`run_sample.py` 는 아래 5.2 와 같은 단계 스크립트를 `--config examples/sample/config.yaml` 로 호출합니다. 샘플 동작을 확인했다면 다음으로 각 단계를 직접 실행해 봅니다.

5.2 단계별 직접 실행

`configs/example.yaml` 과 더미 데이터로 각 단계를 직접 실행합니다. 모든 명령은 **저장소 루트에서** 실행합니다 (YAML 의 상대 경로가 현재 디렉토리 기준이기 때문).

`configs/example.yaml` 은 운영 수준 설정(모델 4종, 모델당 Optuna 30 trials)이라 5.1 샘플보다 오래 걸립니다 — 4,000행 더미 데이터 기준 약 9분 (측정 환경 기준, 대부분 CatBoost 튜닝). 빨리 확인만 하려면 `tuning.n_trials` 를 줄이세요.

```
# 1) 더미 데이터 생성 — data/ 에 train/test/score_input parquet 생성
python examples/make_dummy_data.py

# 2) 학습 (전처리 → 학습 → 후처리)
python train/preprocess.py  --config configs/example.yaml
python train/train.py       --config configs/example.yaml
python train/postprocess.py --config configs/example.yaml
#    한 번에: python train/run_all.py --config configs/example.yaml

# 3) 추론 (전처리 → 예측 → 후처리)
python inference/preprocess.py  --config configs/example.yaml
python inference/predict.py     --config configs/example.yaml
python inference/postprocess.py --config configs/example.yaml
#    한 번에: python inference/run_all.py --config configs/example.yaml

# 4) SHAP 해석 (건별 변수 기여도)
python -m auto_ml.explain.runner --config configs/example.yaml
```

산출물은 다음 구조로 만들어집니다.

```
artifacts/
├── models/
│   ├── best.joblib              ← 전처리기 + 모델 + 메타데이터 한 덩어리
│   └── models/                  ← 후보 모델별 sub-artifact
│       ├── lgbm.joblib
│       ├── xgb.joblib
│       ├── catboost.joblib
│       ├── elasticnet.joblib
```

```
│       └── ensemble.joblib      ← (앙상블이 생성된 경우 — §14)
├── predictions/
│   └── test_predictions.parquet ← 학습 시 holdout 예측 (id + score + prediction +
target)
├── reports/
│   ├── report.html              ← 브라우저로 열기
│   ├── report.pdf               ← 동일 내용 PDF
│   ├── feature_importance.csv   ← 전체 변수 중요도
│   ├── feature_selection.csv    ← (변수 선택 활성 시) 선택 빈도
│   └── sub/                     ← 비-best 모형별 상세 리포트
│       └── <model>/report.html, feature_importance.csv …
├── scores/
│   └── scores.parquet           ← id_columns + score + prediction
├── explanations/
│   └── shap.parquet             ← id_columns + shap_<feature>... + base_value +
score
├── work/                        ← 단계 간 중간 산출물 (5.3 참고)
│   ├── train/
│   │   ├── train_preprocessed.pkl.gz
│   │   └── train_result.pkl.gz
│   └── inference/
│       ├── inference_preprocessed.pkl.gz
│       └── inference_predicted.pkl.gz
└── logs/                        ← 실행한 스크립트별 로그 (B.6 참고)
    ├── train_preprocess_YYYYMMDD_HHMMSS.log
    ├── train_train_YYYYMMDD_HHMMSS.log
    ├── ...
    └── explain_YYYYMMDD_HHMMSS.log
```

`artifacts/reports/report.html` 을 브라우저로 열어 모델 비교표, ROC 곡선, feature importance 등이 표시되면 설치가 정상입니다. 다음 단계는 본인 데이터로 같은 흐름을 돌리는 것입니다 — 필요한 설정은 Part II 에서 섹션별로 찾아보면 됩니다.

자기 데이터로 돌릴 때 보통 손대는 곳은 세 군데입니다.

1. `configs/example.yaml` 을 복사해 데이터 경로·타깃 컬럼 이름만 수정 → § 7 데이터 입력
2. `configs/features.csv` 에 사용할 컬럼을 적기 → § 8 features.csv
3. 필요에 따라 모델·튜닝·앙상블 설정 조정 → § 13 모델, § 12 튜닝, § 14 앙상블

5.3 단계 재실행과 중간 산출물

각 단계 스크립트는 결과를 `work_dir` 에 저장하고, 다음 단계는 그 파일을 읽어 시작합니다. 기본 `work_dir` 은 `artifact_dir` 의 상위 폴더 아래 `work/train` , `work/inference` 입니다 (예: `artifact_dir: ./artifacts/models` → `./artifacts/work/train`). 위치를 바꾸려면 모든 단계에 같은 `--work-dir <경로>` 를 주면 됩니다.

단계	읽는 파일	저장하는 파일
`train/preprocess.py`	학습·테스트 Parquet	`train_preprocessed.pkl.gz` (fit 된 전처리기, 변수 선택 결과, 전처리된 train/test)
`train/train.py`	`train_preprocessed.pkl.gz`	`train_result.pkl.gz` (모델별 학습 결과·지표)
`train/postprocess.py`	위 두 파일	`best.joblib`, 후보 artifact, 리포트, `test_predictions.parquet`
`inference/preprocess.py`	`best.joblib`, 입력 Parquet	`inference_preprocessed.pkl.gz` (모델 입력 행렬, id 컬럼)
`inference/predict.py`	`inference_preprocessed.pkl.gz`, `best.joblib`	`inference_predicted.pkl.gz` (예측 확률, id 컬럼)
`inference/postprocess.py`	`inference_predicted.pkl.gz`	`scores.parquet`

설정을 바꾼 뒤 어느 단계부터 다시 실행하면 되는지는 바꾼 섹션으로 정해집니다.

바꾼 설정	다시 실행할 단계
데이터 경로, `features.csv`, top-level `id_columns`, `preprocessing`, `feature_selection`	`train/preprocess.py` 부터
`training`, `tuning`, `models`, `ensemble`	`train/train.py` 부터
`reporting`	`train/postprocess.py` 만
`scoring.input_path`, `scoring.id_columns`, top-level `id_columns`	`inference/preprocess.py` 부터
`scoring.threshold`, `scoring.output_path`	`inference/postprocess.py` 만
새로 학습했거나 best 를 교체함	`inference/preprocess.py` 부터

주의: 단계 파일에는 "어떤 설정으로 만들었는지" 가 기록되지 않습니다. 앞 단계에 영향을 주는 설정을 바꾸고 뒷 단계만 다시 실행하면, 옛 결과 위에서 실행되므로 위 표를 지켜 주세요. 확실하지 않으면 `run_all.py` 로 전체를 다시 실행하는 것이 안전합니다.

> **더 알아보기 — 실전 예시와 Python API**
>
> - `examples/credit/prepare_data.py` — 신용카드 데이터를 Parquet 으로 변환하는 전처리 예시
> - `examples/titanic/prepare_data.py` — Titanic 데이터셋 전처리 예시
> - Python 코드에서 호출하려면 저장소 루트를 `sys.path` 에 넣은 뒤 다음을 사용합니다.
> - 단계별: `auto_ml.stages.train` 의 `preprocess` / `train` / `postprocess`, `auto_ml.stages.inference` 의 `preprocess` / `predict` / `postprocess`
> - 일괄: `AutoMLPipeline(load_config(path)).run()`, `auto_ml.scoring.runner.run_scoring(config)`

Part II. 설정 레퍼런스

6. 설정 파일 구조와 YAML 문법

6.1 파일 구조

설정 YAML 은 다음 최상위 키들로 구성됩니다. **모든 키는 선택**이며, 누락 시 기본값이 적용됩니다(코드의 `auto_ml/config.py` dataclass 와 1:1 대응).

```
# ----- top-level (데이터 입력) -----
train_data_path: ./data/train.parquet     # §7
test_data_path:  ./data/test.parquet
target_column:   target
features_csv:    ./configs/features.csv
id_columns: [user_id]
artifact_dir:    ./artifacts/models

# ----- 하위 섹션 -----
preprocessing:     {...}    # §9
feature_selection: {...}    # §10
training:          {...}    # §11
tuning:            {...}    # §12
models:            {...}    # §13
ensemble:          {...}    # §14
reporting:         {...}    # §15
scoring:           {...}    # §16
explain:           {...}    # §17
logging:           {...}    # §18
```

전체 옵션을 한눈에 보고 싶다면 `configs/example.yaml` 을 참고하세요 — 모든 기본값이 주석과 함께 노출되어 있어 그대로 복사·수정해서 쓸 수 있습니다.

6.2 YAML 문법 빠른 참고

```yaml
# 1) 스칼라
key: value              # 문자열·숫자·boolean
flag: true              # boolean: true/false
n: 5                    # int
ratio: 0.5              # float
maybe: null             # 명시적 null

# 2) 리스트 (블록 / 인라인)
items:
  - a
  - b
items_inline: [a, b]   # 동일

# 3) 매핑 (중첩)
training:
  cv_folds: 5
  primary_metric: roc_auc

# 4) 인라인 dict — search_space 처럼 한 줄에 여러 키
learning_rate: { type: float, low: 0.01, high: 0.3, log: true }

# 5) 들여쓰기는 항상 공백 2칸 (탭 금지)
# 6) 주석은 #
# 7) 문자열 따옴표는 생략 가능하나 특수문자 포함 시 권장
title: "Credit Default Auto-ML Report"
```

YAML 작성 시 흔히 겪는 함정:

- **탭 들여쓰기 금지** — 모든 들여쓰기는 공백. 에디터 설정에서 "Insert spaces" 켜두기.
- **불리언 표기** — `True` / `False` 도 동작하지만, 표준은 소문자 `true` / `false` .
- **`null` 과 빈 문자열** — `key: null` (값 없음) 과 `key: ""` (빈 문자열) 은 다릅니다.
- **상대경로 기준** — 본 라이브러리의 모든 경로는 **현재 작업 디렉토리(CWD)** 기준입니다. 보통 프로젝트 루트에서 실행하세요.

---

7. 데이터 입력 (top-level)

역할

학습/테스트 데이터의 위치, 타깃 컬럼 이름, 결과에 보존할 식별자를 지정합니다. 본 라이브러리는 학습 데이터를 **자동으로 train/test 분할하지 않습니다** — 사용자가 별도 Parquet 두 개로 준비해야 합니다.

옵션

키	타입	기본값	설명
`train_data_path`	str	`./data/train.parquet`	학습 데이터 Parquet 경로
`test_data_path`	str	`./data/test.parquet`	테스트 데이터 Parquet 경로 (별도 파일)
`target_column`	str	`target`	타깃 컬럼 이름. **값은 0/1 만 허용** (다른 값 섞이면 `ValueError`)
`features_csv`	str	`./configs/features.csv`	사용할 컬럼 정의 CSV 경로. 빈 문자열 `""` 이면 CSV 무시(코드에서 직접 `features` 지정)
`id_columns`	list[str]	`[]`	결과(스코어링/SHAP) 에 그대로 보존할 식별자 컬럼 목록
`artifact_dir`	str	`./artifacts/models`	`best.joblib` 과 sub-artifact 가 저장될 디렉토리

YAML 예시

```yaml
train_data_path: ./data/my_train.parquet
test_data_path:  ./data/my_test.parquet
target_column:   churn
features_csv:    ./configs/my_features.csv
id_columns:
  - user_id
artifact_dir:    ./artifacts/models
```

동작 메모

- **타깃은 0/1 만 허용** — `utils/validation.py` 가 검증합니다. 결측·문자열·2 이상의 정수 등이 섞여 있으면 즉시 실패합니다.

- **컬럼 일관성** — 학습/테스트 데이터에는 모든 feature 컬럼이 있어야 합니다(없으면 에러). 스코어링 입력에서 feature 컬럼이 통째로 빠지면 에러 대신 학습 시 기본값(수치형 median, 범주형 최빈값)으로 채우고 WARNING 을 남깁니다(§16 참고).
- **`id_columns`** 는 top-level 에서 한 번만 적어두면 `scoring.id_columns` / `explain.id_columns` 가 비어 있을 때 자동 fallback 됩니다.

---

8. features.csv — 사용할 컬럼 정의

역할

데이터의 어떤 컬럼을 어떤 타입으로 학습에 쓸지 명시하는 명단입니다. **운영 중 변수를 켜고 끌 때 코드를 고치지 않고 이 CSV 만 수정**하면 됩니다.

CSV 형식

컬럼	의미	허용 값
`name`	Parquet 의 컬럼명과 정확히 일치	(Parquet 컬럼명)
`type`	컬럼 종류	`continuous` (수치형) / `category` (범주형)
`used`	학습/스코어링 사용 여부	`true` / `false` (대소문자 무시, `1` / `0`, `yes` / `no` 도 허용)

예시

```
name,type,used
age,continuous,true
gender,category,true
income,continuous,true
internal_score,continuous,false
```

동작 메모

- **타깃 컬럼·id 컬럼은 적지 않습니다.** 각각 `target_column` / `id_columns` 로 따로 지정합니다.
- `used: false` 인 행은 학습/스코어링에 포함되지 않습니다 (CSV 에 남겨두기만 해도 무방).
- 빈 행은 조용히 건너뜁니다. 같은 `name` 이 두 번 나오면 `ValueError`.
- 결과적으로 `used == true` 인 행이 하나도 없으면 `ValueError` (학습할 컬럼이 없음).
- 숫자 코드(예: 지역코드 1, 2, 3)도 범주로 쓰려면 `category` 로 적어야 합니다. 자동 판별은 하지 않습니다. 범주형 값은 전처리에서 문자열로 정규화되므로 `1` / `1.0` / `"1"` 은 같은 범주로 취급됩니다 (§9 동작 메모 참고).

---

9. 전처리 `preprocessing`

역할

전처리는 **항상** 다음 순서로 적용됩니다.

```
(1) 결측 처리  →  (2) 이상치 처리  →  (2.5) skew 변환  →  (3) 스케일링
```

학습 시 산출된 통계량(중앙값, 분위수 경계, 분위수 테이블, 스케일러 fit 결과)은 모두 `best.joblib` 에 저장되어 스코어링 시 **그대로 재적용**됩니다 — 학습/스코어링 일관성이 자동 보장됩니다.

옵션

키	타입	기본값	허용 값	설명
`numeric_null_strategy`	str	`median`	`median` / `mean` / `constant`	수치형 결측 채움 전략
`numeric_null_fill_value`	float	`0.0`	—	`constant` 일 때 채움 값
`categorical_null_strategy`	str	`most_frequent`	`most_frequent` / `constant`	범주형 결측 채움 전략
`categorical_null_fill_value`	str	`MISSING`	—	`constant` 일 때 채움 값
`outlier_method`	str	`percentile`	`percentile` / `none`	이상치 처리 방식
`outlier_lower_quantile`	float	`0.01`	`[0, 1)`	윈저라이징 하한 분위수
`outlier_upper_quantile`	float	`0.99`	`(0, 1]`	윈저라이징 상한 분위수
`outlier_action`	str	`clip`	`clip` / `null_then_impute`	경계 밖 값 처리 방식
`skew_method`	str	`signed_log1p`	`signed_log1p` / `quantile_normal` / `none`	skew 변환 방식
`skew_threshold`	float	`1.0`	—	`|skew| > threshold` 인 컬럼만 변환

키	타입	기본값	허용 값	설명
`scaling_method`	str	`standard`	`standard` / `minmax` / `robust` / `none`	스케일링 방식

YAML 예시

```
preprocessing:
  numeric_null_strategy: median               # median | mean | constant
  numeric_null_fill_value: 0.0
  categorical_null_strategy: most_frequent    # most_frequent | constant
  categorical_null_fill_value: MISSING

  outlier_method: percentile                  # percentile | none
  outlier_lower_quantile: 0.01
  outlier_upper_quantile: 0.99
  outlier_action: clip                        # clip | null_then_impute

  skew_method: signed_log1p
# signed_log1p | quantile_normal | none
  skew_threshold: 1.0

  scaling_method: standard                    # standard | minmax | robust |
none
```

동작 메모

결측 처리 - 전부-NaN 가드: 학습 데이터에서 어떤 수치형 컬럼이 전부 NaN 이면 `median` / `mean` 계산이 불가능해 silent 전파를 막기 위해 `ValueError` 로 실패합니다. `features.csv` 에서 해당 컬럼을 `used: false` 로 두거나 `constant` 전략으로 전환하세요. - 스코어링 시 컬럼 자체가 통째로 누락된 경우에도 학습 시점의 기본값(수치형 median, 범주형 최빈값)으로 자동 채워집니다 — 일시 누락에 대한 안전망.

범주형 값 정규화 - 결측 처리 전에 범주형 값을 같은 규칙의 문자열로 맞춥니다: 정수·정수값 실수(`1` , `1.0`) → `"1"` , 그 외 실수 → `"1.5"` , bool → `"True"` / `"False"` , 문자열은 그대로. - 그래서 학습 데이터에 결측이 있어 컬럼이 실수(`1.0`)로 읽히고 추론 데이터는 정수(`1`)로 들어와도 같은 범주로 인식됩니다. 결측 대체값도 같은 규칙으로 만들어져 타입이 섞이지 않습니다. - 문자열은 손대지 않으므로 `"A"` 와 `"a "` (대소문자·공백 차이) 는 다른 범주입니다. - 정수 코드는 자릿수 그대로 문자열이 됩니다(아주 큰 코드도 정확). 단, 2^53(약 9×10^15)보다 큰 정수 코드가 결측과 섞인 컬럼은 pandas 가 읽을 때 실수로 바꾸면서 이미 정밀도를 잃으므로, 이런 코드(계좌번호처럼 긴 ID 성 코드)는 데이터에 **문자열로** 저장하세요. - 학습 때 없던 범주가 test / 추론 / SHAP 입력에 나오면 컬럼별 비율을 로그로 남깁니다 (1% 이상 WARNING, 그 미만 INFO). 처리 방식은 모델마다 다릅니다 — XGBoost·LightGBM 은 결측과 같게, ElasticNet 은 모든 One-hot 열 0, CatBoost 는 자체 규칙. - 이 기능 이전에 학습된 `best.joblib` 은 기존 방식으로 동작합니다 (재학습 전까지 결과 동일).

이상치 처리 - `clip` (기본): 경계 밖 값을 경계로 잘라냅니다. - `null_then_impute` : 경계 밖을 NaN 으로 바꾼 뒤 median 으로 다시 채웁니다. - `lower < upper` 위반 시 `ValueError` .

Skew 변환 - 변환 대상은 학습 시 한 번 결정됩니다 (`|skew| > skew_threshold` 인 컬럼). 미선택 컬럼은 스코어링에서도 통과하여 분포 일관성이 유지됩니다. - `signed_log1p` : `sign(x) * log1p(|x|)` . 무상태 (추가 저장 없음), 음수 보존. - `quantile_normal` : 학습 분포의 분위수를 표준정규로 매핑. 이상치/skew 모두에 강건하지만 분위수 테이블이 artifact 에 저장됩니다(크기 약간 증가). - 부스팅 트리만 쓰면 효과가 작으므로 `none` 으로 꺼도 무방합니다.

스케일링 - 부스팅 트리는 스케일링에 본질적으로 무관합니다. 옵션을 두는 이유는 선형/신경망 모델 확장을 대비함이며, 트리만 쓴다면 `none` 으로 둬도 됩니다.

---

10. 변수 선택 `feature_selection`

역할

학습 데이터의 부분표본을 여러 번 뽑아 각 부분표본에서 변수를 골라 보고, **자주 선택되는 변수만** 최종 채택합니다 (Meinshausen & Bühlmann, 2010 의 Stability Selection). 단일 fit 한 번의 우연성을 제거해 noise 변수가 채택되는 위험을 줄입니다.

기본값은 `enabled: false` — 변수가 적으면 (예: < 30개) 부스팅 모델 내장 selection 으로 충분하므로 굳이 켜지 않아도 됩니다.

옵션

키	타입	기본값	허용 값	설명
`enabled`	bool	`false`	—	사용 여부
`base_estimator`	str	`lasso`	`lasso` / `lgbm`	부분표본 단위 선택기
`n_subsamples`	int	`200`	—	부분표본 추출 횟수
`subsample_ratio`	float	`0.5`	`(0, 1)`	부분표본 크기 비율
`threshold`	float	`0.6`	`[0, 1]`	채택 임계값 (선택 빈도)
`random_state`	int	`42`	—	재현성 시드
`min_selected`	int	`1`	—	임계값 미달 시 빈도 상위 N 개 fallback
`lasso_C`	float	`0.1`	—	(lasso 전용) L1 규제 강도
`lgbm_top_k`	int	`30`	—	(lgbm 전용) 부분표본당 gain 상위 K
`lgbm_n_estimators`	int	`100`	—	(lgbm 전용) 부분표본 학습용 트리 수

키	타입	기본값	허용 값	설명
`lgbm_learning_rate`	float	`0.1`	—	(lgbm 전용) 부분표본 학습률

YAML 예시

```yaml
feature_selection:
  enabled: true
  base_estimator: lasso
  n_subsamples: 200
  subsample_ratio: 0.5
  threshold: 0.6
  random_state: 42
  min_selected: 3

  # base_estimator: lasso 일 때만 사용
  lasso_C: 0.1

  # base_estimator: lgbm 일 때만 사용
  lgbm_top_k: 30
  lgbm_n_estimators: 100
  lgbm_learning_rate: 0.1
```

동작 메모

`base_estimator` 선택 기준 비교

항목	`lasso` (L1 로지스틱)	`lgbm` (LightGBM gain)
선택 기준	non-zero coefficient	gain 상위 K (>0 만)
잘 잡는 신호	선형 효과	비선형 · 상호작용
범주형 처리	full-X frequency encoding (사전 1회)	pandas `category` dtype native
부분표본당 속도	빠름	상대적으로 느림

설정 가이드 - `n_subsamples` — 표준 100~500. 작은 데이터면 50~100 도 충분, 큰 데이터·고차원이면 200+. - `subsample_ratio` — 원 논문 권고 0.5. 너무 크면 모든 부분표본이 비슷해져 안정화 안 됨, 너무 작으면 fit 자체가 흔들림. - `threshold` — 0.6 보수적, 0.8 매우 보수적. false-positive 통제 강도의 trade-off. - `min_selected` — 채택이 부족할 때 발동하는 안전망. 평소엔 발동되지 않아야 하고, 발동 시 WARN 로그가 남습니다. - `lasso_C` — 낮을수록 규제 강함(더 sparse). 0.1 시작, 변수가 너무 많이 살면 0.01, 너무 잘려나가면 1.0. - `lgbm_top_k` — 전체 feature 의 30~50% 정도가 적당. 너무 크면 threshold 통과 변수가 폭증.

산출물 - 채택된 변수 목록·선택 빈도·fallback 발동 여부가 `SelectionResult` 로 `best.joblib` 메타데이터에 저장됩니다. - 스코어링 시 동일 컬럼 셋만 모델로 흘러갑니다 — 일관성 자동 보장. - HTML/PDF 리포트에 변수별 빈도 막대와 채택/제외 라벨이 표기됩니다.

---

11. 학습 `training`

역할

CV 폴드 수, 조기 종료 라운드, 모델 선택 지표, **최종 모델 학습 전략** 을 결정합니다. 파이프라인은 각 모델에 대해 (1) 튜닝 → (2) StratifiedKFold OOF 평가 → (3) 최종 fit → (4) 테스트 평가의 순서로 동작하며, best 모델은 테스트 점수 기준으로 자동 선정됩니다.

옵션

키	타입	기본값	허용 값	설명
`cv_folds`	int	`5`	—	최종 OOF 평가용 StratifiedKFold 분할 수
`random_state`	int	`42`	—	폴드 분할 등 재현성 시드
`early_stopping_rounds`	int	`50`	—	부스팅 모델 조기 종료 라운드 수
`primary_metric`	str	`roc_auc`	`roc_auc` / `pr_auc` / `ks` / `f1` / `accuracy` / `precision` / `recall` / `lift`	모델 선택·튜닝 목적함수
`best_model`	str / null	`null`	`null` / `lgbm` / `xgb` / `catboost` / `elasticnet` / `ensemble`	best 강제 지정 (null = 자동)
`final_fit_strategy`	str	`early_stop_on_test`	`early_stop_on_test` / `iteration_capping` / `cv_bagging`	최종 모델 학습 방식
`iteration_cap_aggregation`	str	`mean`	`mean` / `median`	(capping 전용) fold best_iter 집계 방식

키	타입	기본값	허용 값	설명
`iteration_cap_headroom`	float	`1.0`	`> 0`	(capping 전용) 집계값 × headroom 후 ceil
`n_jobs`	int / null	`null`	`null` / 양수 / 음수	학습(튜닝·CV·변수 선택) CPU 스레드 수. null·`-1` = **노드 전체 코어 (팟 CPU limit 아님)**, `-2` = 하나 남김. 팟에서는 양의 정수로 명시 (§ 20)

`primary_metric` 가이드:

지표	추천 도메인
`roc_auc`	일반 (기본)
`pr_auc`	positive 비율이 매우 낮은 불균형 데이터
`ks`	신용평점 / 리스크 모델링
`f1` / `precision` / `recall`	임계값 0.5 기준 평가
`lift`	상위 분위 lift

YAML 예시 (기본 옵션)

```
training:
  cv_folds: 5
  random_state: 42
  early_stopping_rounds: 50
  primary_metric: roc_auc
  best_model: null
```

> 최종 모델 학습 전략(`final_fit_strategy` / `iteration_cap_*`)은 옵션이 많아 아래 최종 모델 학습 전략 서브섹션에서 따로 다룹니다.

동작 메모

`early_stopping_rounds` 는 부스팅 세 모델에 통합 적용되지만 내부 구현이 다릅니다 (LGBM callbacks, XGB 생성자 인자, CatBoost 학습 인자). OOF fold 학습에는 항상 적용되지만, **최종 fit 의 적용 여부는 `final_fit_strategy` 에 좌우**됩니다 (아래 서브섹션 참고).

`best_model` 강제 지정 - `lgbm` / `xgb` / `catboost` / `elasticnet` / `ensemble` 중 하나로 지정 가능. 알 수 없는 이름이면 `ValueError`. - 학습 후 변경하려면 `python -m auto_ml.cli.set_best --model <name>` 사용 (재학습 불필요).

최종 모델 학습 전략 (`final_fit_strategy`)

`final_fit_strategy` 는 OOF 평가가 끝난 뒤 **운영에 쓸 최종 모델을 어떻게 만들지**를 결정합니다. 세 가지 중 하나를 고르며, 기본값은 현행 동작과 동일한 `early_stop_on_test` 입니다 — **지정하지 않으면 동작이 바뀌지 않습니다.**

왜 필요한가

기본 `early_stop_on_test` 는 최종 모델을 학습할 때 **테스트셋을 early-stopping 검증셋으로 사용**합니다. 편리하지만 최종 트리 수(`best_iter`)와 best 모델 선정이 테스트셋에 맞춰지므로, 리포트 §3 "오버핏 점검" 의 `Δ = Train − Test` 가 실제보다 **작게(낙관적으로)** 나옵니다 — 즉 테스트 점수가 약간 부풀려집니다. 아래 두 전략은 학습 단계에서 테스트셋을 전혀 보지 않아 Δ 가 정직해집니다.

세 전략 비교

전략	동작	테스트셋 노출	언제 쓰나
`early_stop_on_test` (기본)	학습 전체로 fit, 테스트셋을 조기종료 검증셋으로 사용	O	현행 호환 / 단일 데이터셋 일관성 우선
`iteration_capping`	CV fold best_iter 집계로 트리 수 고정, early stop 없이 train 전체로 재학습	X	오버핏 Δ 가 큰데 정직한 트리 수로 재학습하고 싶을 때
`cv_bagging`	최종 재학습 생략, CV 의 K fold 모델을 균등 가중 앙상블로 사용	X	분산을 줄이고 안정적인 모델을 원할 때 (재학습 비용도 절약)

관련 옵션 (모두 `training:` 아래):

키	적용 전략	설명
`final_fit_strategy`	공통	`early_stop_on_test` (기본) / `iteration_capping` / `cv_bagging`

키	적용 전략	설명
`iteration_cap_aggregation`	iteration_capping	fold best_iter 집계 방식. `mean` (기본) / `median`
`iteration_cap_headroom`	iteration_capping	집계값에 곱하는 여유 배수. `capped = ceil(agg × headroom)` . 기본 1.0

YAML 예시

오버핏 Δ 가 큰 경우 — 정직한 트리 수로 재학습:

```
training:
  cv_folds: 5
  primary_metric: roc_auc
  final_fit_strategy: iteration_capping
  iteration_cap_aggregation: mean     # 각 fold best_iter 의 평균 (median 도 가능)
  iteration_cap_headroom: 1.0         # 1.0 = 평균 그대로. 1.05~1.1 로 올리면 약간 더
학습
```

안정적인 앙상블 — CV fold K 개 모델을 균등 평균:

```
training:
  final_fit_strategy: cv_bagging
```

동작 메모

- 두 신규 전략은 학습 단계에서 테스트셋을 보지 않으므로 리포트 §3 의 Δ 가 정직해집니다 (기본 전략보다 약간 커 보일 수 있는데, 편향이 제거된 정상 수치입니다).
- **ElasticNet 은 `iteration_capping` 에서 자동 면제** (부스팅이 아니라 고정할 트리 수가 없음). 일반 refit 으로 동작하며 INFO 로그가 남습니다.
- **`cv_bagging` 과 `ensemble.enabled: true` 를 함께 켜면** 외부 앙상블이 자동 비활성화되고 WARN 로그가 남습니다 (각 모델이 이미 fold 평균 = bagging 이므로 중복 방지).
- 사용된 전략은 `<artifact_dir>/models/<name>.joblib` 메타데이터의 `extra.training_mode` 에 기록됩니다.

---

12. 튜닝 `tuning`

역할

Optuna TPE 샘플러로 각 모델의 `search_space` 를 탐색해 `primary_metric` KFold OOF 평균을 최대화하는 하이퍼파라미터를 찾습니다. 본 단계는 **테스트 데이터를 절대 사용하지 않습니다** (정보 누설 방지).

옵션

키	타입	기본값	설명
`enabled`	bool	`true`	False 면 모든 모델 튜닝 생략
`n_trials`	int	`30`	모델당 시도 횟수
`timeout`	int / null	`null`	모델당 wall-clock 상한(초). null = 무제한
`cv_folds`	int	`3`	튜닝 단계 KFold 분할 수 (보통 `training.cv_folds` 보다 작게)
`random_state`	int	`42`	TPE 샘플러 시드

YAML 예시

```
tuning:
  enabled: true
  n_trials: 30
  timeout: 1800        # 모델당 30분 상한
  cv_folds: 3
  random_state: 42
```

동작 메모

- 모델별 `search_space` 가 비어 있으면 그 모델은 `fixed_params` 만으로 학습하고 튜닝을 생략합니다.
- 튜닝 단계의 `early_stopping_rounds` 는 `training.early_stopping_rounds` 를 그대로 사용합니다.
- **빠른 1차 검증**: `n_trials: 10` , `cv_folds: 2` . **운영 권장**: `n_trials: 30~50` , `cv_folds: 3` , 필요 시 `timeout` 으로 wall-clock 보호.

---

13. 모델 `models`

역할

활성화할 모델과 각 모델의 하이퍼파라미터 / 탐색 공간 / 손실 함수를 지정합니다. 모든 모델은 동일 인터페이스(`fit / predict_proba / feature_importance / shap_values`)를 따르므로 학습/ 스코어링/SHAP 경로가 모델 종류에 무관합니다.

13.1 공통 구조

`models.<name>` 의 키 (모든 모델 공통):

키	타입	기본값	설명
`enabled`	bool	`true` (ElasticNet 만 `false`)	학습에 포함 여부
`fixed_params`	dict	`{}`	항상 적용되는 라이브러리 원본 파라미터
`search_space`	dict	`{}`	Optuna 탐색 공간. 비어 있으면 튜닝 생략
`loss`	str	`logloss`	`logloss` (기본) / `focal` (Focal Loss)

`search_space` 항목 형식:

```
# 1) float — low/high 필수, log 선택(기본 false)
learning_rate: { type: float, low: 0.01, high: 0.3, log: true }

# 2) int — low/high 필수, log·step 선택(step 기본 1)
num_leaves:    { type: int,   low: 15,   high: 255 }

# 3) categorical — choices 필수
boosting_type: { type: categorical, choices: [gbdt, dart] }
```

13.2 LightGBM `lgbm`

- 범주형 처리: pandas `category` dtype native
- early stopping: `callbacks=[early_stopping(rounds)]`
- Focal Loss: 지원

```
models:
  lgbm:
    enabled: true
    fixed_params:
      objective: binary
      metric: auc
      verbose: -1
      n_estimators: 2000
    search_space:
      learning_rate:    { type: float, low: 0.01,   high: 0.3,  log: true }
      num_leaves:       { type: int,   low: 15,     high: 255 }
      max_depth:        { type: int,   low: 3,      high: 12 }
      min_data_in_leaf: { type: int,   low: 10,     high: 100 }
      feature_fraction: { type: float, low: 0.6,    high: 1.0 }
      bagging_fraction: { type: float, low: 0.6,    high: 1.0 }
      reg_alpha:        { type: float, low: 1.0e-8, high: 10.0, log: true }   #
L1
      reg_lambda:       { type: float, low: 1.0e-8, high: 10.0, log: true }   #
```

```
L2
      min_gain_to_split:{ type: float, low: 0.0,    high: 0.5 }
```

13.3 XGBoost `xgb`

- 범주형 처리: native (`enable_categorical=True`). 학습 때 본 범주 목록으로 고정한 pandas `category` 로 넘기므로 범주 사이에 순서를 가정하지 않고, 학습 때 없던 범주는 결측으로 처리합니다. XGBoost 2.x 가 필요합니다 (2.1.2 에서 검증).
- 이 기능 이전에 학습된 artifact 는 기존 `LabelEncoder` 방식(unseen 은 -1) 으로 예측합니다.
- early stopping: 생성자 인자 `early_stopping_rounds` (XGB 2.x sklearn API)
- Focal Loss: 지원

```
models:
  xgb:
    enabled: true
    fixed_params:
      objective: binary:logistic
      eval_metric: auc
      tree_method: hist
      n_estimators: 2000
    search_space:
      learning_rate:    { type: float, low: 0.01,   high: 0.3,  log: true }
      max_depth:        { type: int,   low: 3,      high: 10 }
      subsample:        { type: float, low: 0.6,    high: 1.0 }
      colsample_bytree: { type: float, low: 0.6,    high: 1.0 }
      min_child_weight: { type: float, low: 1.0,    high: 10.0 }
      reg_alpha:        { type: float, low: 1.0e-8, high: 10.0, log: true }   #
L1
      reg_lambda:       { type: float, low: 1.0e-8, high: 10.0, log: true }   #
L2
      gamma:            { type: float, low: 0.0,    high: 5.0 }
```

13.4 CatBoost `catboost`

- 범주형 처리: native (`cat_features` 인덱스 전달)
- early stopping: 학습 인자 `early_stopping_rounds`
- iteration 키: **`iterations`** (다른 모델의 `n_estimators` 와 다름)
- Focal Loss: 지원 (`PythonUserDefinedObjective`)
- L1 정규화: **미지원** (L2 만 사용 — `l2_leaf_reg`)

```
models:
  catboost:
    enabled: true
    fixed_params:
      loss_function: Logloss
```

```
    eval_metric: AUC
    iterations: 2000              # ← n_estimators 아님!
    verbose: false
    allow_writing_files: false     # 권장 (작업 디렉토리 오염 방지)
  search_space:
    learning_rate:       { type: float, low: 0.01, high: 0.3, log: true }
    depth:               { type: int,   low: 4,    high: 8 }
    l2_leaf_reg:         { type: float, low: 1.0,  high: 100.0, log: true }
    random_strength:     { type: float, low: 0.0,  high: 10.0 }
    bagging_temperature: { type: float, low: 0.0,  high: 1.0 }
```

13.5 ElasticNet `elasticnet`

- sklearn `LogisticRegression(solver='saga')` 기반 L1+L2 혼합 규제 선형 모델
- 범주형 처리: 내부 `OneHotEncoder` (학습 시 fit, `handle_unknown="ignore"`)
- early stopping: **무시** (sklearn 에 해당 개념 없음)
- iteration capping: **자동 면제** (부스팅 아님)
- Focal Loss: **미지원** (`loss: logloss` 만)
- 기본값 `enabled: false` — 필요 시 활성화

```
models:
  elasticnet:
    enabled: true
    fixed_params:
      max_iter: 2000        # 수렴이 안 되면 늘리기
    search_space:
      C:        { type: float, low: 1.0e-3, high: 10.0, log: true }
      l1_ratio: { type: float, low: 0.0,    high: 1.0 }
```

- `C` — 역규제 강도. 낮을수록 규제 강함(더 sparse).
- `l1_ratio` — 0.0 = 순수 Ridge (L2), 1.0 = 순수 Lasso (L1). 중간값은 혼합.

13.6 Focal Loss (선택)

클래스 불균형이 큰 데이터(예: positive 비율 < 5%)에서 `loss: focal` 로 전환 가능. 모델별 독립이며, LGBM/XGB/CatBoost 만 지원합니다.

```
models:
  xgb:
    loss: focal                   # logloss (기본) | focal
    fixed_params:
      eval_metric: auc            # rank-기반 metric 필수 (auc, pr_auc)
      tree_method: hist
      n_estimators: 2000
      focal_gamma: 2.0            # easy-example 감쇠 지수 (기본 2.0)
```

```
    focal_alpha: 0.25             # 양성 클래스 가중 (기본 0.25)
  search_space:
    # focal 하이퍼파라미터도 탐색 가능:
    # focal_gamma: { type: float, low: 0.5, high: 4.0 }
    # focal_alpha: { type: float, low: 0.1, high: 0.9 }
```

- 호환 metric: **rank-based** (`auc` , `pr_auc` , `AUC`). `binary_logloss` / `Logloss` 등 절대값 기반은 raw score 에 잘못 적용되어 의미가 깨지므로 사용 금지.
- `focal_gamma` / `focal_alpha` 는 wrapper 가 pop 하므로 라이브러리 원본 인자에 전달되지 않습니다 (예약 키).

---

14. 앙상블 `ensemble`

역할

활성화된 개별 모델 학습 완료 후 추가로 앙상블 모델을 생성합니다. best 선정 시 앙상블도 후보에 포함됩니다.

옵션

키	타입	기본값	허용 값	설명
`enabled`	bool	`true`	—	앙상블 생성 여부 (활성 모델 ≥ 2개 필요)
`strategy`	str	`elasticnet_plus_best`	`elasticnet_plus_best` / `weighted_average`	구성 방식
`elasticnet_weight`	float / null	`null`	`(0, 1)` 또는 null	(전자 전용) elasticnet 가중치, null = 자동

YAML 예시

기본 (ElasticNet + 나머지 best):

```
ensemble:
  enabled: true
  strategy: elasticnet_plus_best
```

```
  elasticnet_weight: null           # 점수 비례 자동 계산
  # elasticnet_weight: 0.35          # 0~1 사이 값으로 직접 지정도 가능
```

전체 모델 점수 비례 가중 평균:

```
ensemble:
  enabled: true
  strategy: weighted_average
```

동작 메모

전략별 동작 - `elasticnet_plus_best` — `elasticnet` 모델을 무조건 포함하고 나머지 활성 모델 중 test `primary_metric` 1위 1개와 결합. ElasticNet 이 비활성이면 앙상블을 만들지 않고 `Ensemble skipped — ...` WARNING 만 남긴 채 학습을 계속합니다. - `weighted_average` — 활성화된 모든 모델을 test `primary_metric` 점수에 비례하는 가중치로 결합: `weight[i] = score[i] / sum(score)` .

> **주의 — YAML 에 `elasticnet` 블록이 없으면 앙상블이 만들어지지 않습니다.** 기본 strategy 는 `elasticnet_plus_best` 인데 ElasticNet 의 코드 기본값은 `enabled: false` 입니다. 이 경우 학습은 계속되고 `Ensemble skipped — ...` WARNING 만 남습니다. 앙상블을 쓰려면 `models.elasticnet.enabled: true` 를 두거나 `strategy: weighted_average` 로 바꾸세요. 저장소의 예시 설정(`configs/example.yaml` , `examples/credit/config.yaml` , `examples/sample/config.yaml`)은 모두 ElasticNet 을 켜 두었습니다. 직접 만든 설정에서 `elasticnet` 블록을 지우면 이 경고가 나옵니다.

호환성 - 앙상블의 SHAP 은 서브모델마다 확률 도메인으로 바꾼 뒤 앙상블과 같은 가중치로 평균합니다. 그래서 SHAP 결과의 `score` 가 스코어링 점수와 정확히 일치합니다 (`cv_bagging` 모델도 동일). - 앙상블의 feature importance 는 서브모델별 중요도를 합 1 인 비율로 맞춘 뒤 앙상블 가중치로 합산합니다 (모델마다 중요도 단위가 달라 그대로 더하면 한 모델이 전부를 차지하기 때문). 그래서 앙상블 리포트 CSV 의 `importance_gain` 컬럼은 gain 이 아니라 비율 값입니다. - `cv_bagging` 과 함께 켜면 외부 앙상블은 **자동 비활성화** + WARN (각 모델이 이미 fold 평균). - 앙상블이 best 로 선정되면 `best.joblib` 이 곧 앙상블입니다. cloudpickle 이 서브모델 전체 객체 그래프를 직렬화하므로 단일 파일로 스코어링·SHAP 모두 동작합니다.

---

15. 리포트 `reporting`

역할

HTML / PDF 동일 내용 리포트를 생성합니다(Jinja2 + WeasyPrint). 모델 비교표, CV(OOF) 비교, **오버핏 점검 (Train vs Test, Δ)**, 튜닝 결과, ROC / PR 곡선, feature importance, score 분포, confusion matrix, (활성 시) 변수 선택 결과 등을 포함합니다.

옵션

키	타입	기본값	설명
`output_dir`	str	`./artifacts/reports`	리포트와 부속 CSV 저장 디렉토리
`generate_html`	bool	`true`	HTML 생성 여부
`generate_pdf`	bool	`true`	PDF 생성 여부 (WeasyPrint 필요)
`title`	str	`Auto-ML Binary Classification Report`	리포트 제목
`top_importance_features`	int / null	`30`	feature importance 표·차트에 노출할 상위 N개. null = 전체
`top_selection_features`	int / null	`null`	변수 선택 결과 노출 상위 N개. null = 전체

YAML 예시

```
reporting:
  output_dir: ./artifacts/credit/reports
  generate_html: true
  generate_pdf: true
  title: Credit Card Default Auto-ML Report
  top_importance_features: 30
  top_selection_features: null
```

동작 메모

- WeasyPrint 시스템 의존성이 없으면 PDF 생성에 실패합니다. 이때 학습은 중단되지 않습니다 — 모델 (`best.joblib` 등)은 리포트보다 먼저 저장되고, HTML·CSV 도 정상 생성되며, 로그에 `PDF report skipped` WARNING 이 한 번 남습니다. PDF 가 필요 없으면 `generate_pdf: false` 로 끄고, 필요하면 §3 의 시스템 패키지를 설치하세요.
- 부속 CSV (`feature_importance.csv` , `feature_selection.csv`) 는 **항상 전체 변수**를 저장합니다 (`top_*` 옵션은 HTML 표시에만 영향).
- best 가 아닌 모형마다 별도 sub-report 가 자동 생성됩니다 (`output_dir/sub/<name>/report.{html,pdf}`).

---

16. 스코어링 `scoring`

역할

추론(`inference/*.py`) 이 사용하는 옵션. 학습된 `best.joblib` 으로 새 데이터에 점수를 매겨 Parquet 으로 출력합니다.

옵션

키	타입	기본값	설명
`input_path`	str	`./data/score_input.parquet`	스코어링 대상 Parquet 경로
`output_path`	str	`./artifacts/scores/scores.parquet`	결과 저장 경로
`id_columns`	list[str]	`[]`	결과에 보존할 식별자 (override)
`threshold`	float	`0.5`	`predict_proba` → 0/1 변환 임계값
`n_jobs`	int / null	`null`	추론·SHAP CPU 스레드 수. null = 학습 때 값 그대로(학습도 null 이면 노드 전체 코어). 규칙은 `training.n_jobs` 와 같음. 예측 결과는 바뀌지 않음. 추론 팟 CPU 가 학습 팟과 다르면 따로 명시 (§ 20)

YAML 예시

```
scoring:
  input_path:  ./data/score_input.parquet
  output_path: ./artifacts/scores/scores.parquet
  id_columns:
    - user_id
  threshold: 0.5
```

동작 메모

- 결과 컬럼: `<id_columns> + score + prediction` .
- `id_columns` 우선순위: (1) `scoring.id_columns` → (2) top-level `id_columns` → (3) artifact 메타데이터에 저장된 학습 시점 id_columns. 보통 top-level 에 한 번 적어두면 충분합니다.
- 스코어링 입력에는 학습 시점의 feature 컬럼이 있어야 합니다. 컬럼이 통째로 빠지면 에러가 아니라 학습 시 기본값(수치형 median, 범주형 최빈값)으로 채우고 WARNING 을 남깁니다 — 일시 누락 대비 안전망이므로, 이 경고가 반복되면 입력 데이터를 점검하세요.

- `threshold` 는 도메인에 따라 조정 (예: false-negative 비용이 크면 0.3 으로 낮춤).

---

17. SHAP 해석 `explain`

역할

SHAP 해석(`python -m auto_ml.explain.runner`) 이 사용하는 옵션. 학습된 모델이 각 행을 왜 그렇게 예측했는지 **건별·변수별 기여도(SHAP)** 를 산출해 Parquet 으로 저장합니다. LGBM/XGB/CatBoost 는 native API (`pred_contrib`), ElasticNet 은 `shap.LinearExplainer` , 앙상블은 서브모델별 확률 도메인 SHAP 의 가중 평균을 사용합니다. 어떤 모델이든 출력의 `score` 는 스코어링 점수와 같습니다.

옵션

키	타입	기본값	설명
`input_path`	str	`./data/score_input.parquet`	해석 대상 Parquet (보통 스코어링 입력과 동일)
`output_path`	str	`./artifacts/explanations/shap.parquet`	결과 저장 경로
`id_columns`	list[str]	`[]`	결과 식별자 (override). 스코어링과 동일 우선순위

YAML 예시

```
explain:
  input_path:  ./data/score_input.parquet
  output_path: ./artifacts/explanations/shap.parquet
  id_columns:
    - user_id
```

출력 스키마

```
<id_columns> + shap_<feature_1> + shap_<feature_2> + ... + base_value + score
```

- `base_value` : 모델의 평균 예측 확률 (각 행 동일).
- `shap_<feature>` : 해당 변수가 평균 대비 예측을 얼마나 끌어올렸/내렸는지 (**확률 도메인**).
- `score` : 최종 예측 확률 (= `scoring` 의 `score` 와 동일).
- **가법성 보장**: 모든 행에서 `base_value + sum(shap_*) ≈ score` (오차 ≤ 1e-12).

동작 메모

native API 는 raw-margin (logit) 도메인 SHAP 을 반환합니다. 본 라이브러리는 다음과 같이 확률 도메인으로 변환하면서 가법성을 보존합니다.

```
p_base = sigmoid(base_raw)
p_pred = sigmoid(base_raw + sum(feature_raw))
shap_prob[i] = feature_raw[i] / sum(feature_raw) * (p_pred - p_base)
```

비율은 raw 도메인 그대로 유지되고, 합산만 `(score - base_value)` 와 정확히 일치하도록 스케일링됩니다.

---

18. 로깅 `logging`

역할

학습/추론/SHAP 스크립트를 실행할 때마다 별도 로그 파일을 자동 생성합니다.

옵션

키	타입	기본값	허용 값	설명
`log_dir`	str	`./artifacts/logs`	—	파일 로그 저장 디렉토리
`level`	str	`INFO`	`DEBUG` / `INFO` / `WARNING` / `ERROR`	로깅 레벨
`to_stdout`	bool	`true`	—	콘솔 출력 여부
`to_file`	bool	`true`	—	파일 출력 여부

YAML 예시

```
logging:
  log_dir: ./artifacts/logs
  level: INFO
  to_stdout: true
  to_file: true
```

동작 메모

- 파일명은 `<stage>_YYYYMMDD_HHMMSS.log` 형식이며, `<stage>` 는 실행한 스크립트에 따라 정해집니다 (목록은 B.6 로그 파일).
- 콘솔과 파일은 동시에 활성화 가능. `to_file: false` 면 파일은 만들어지지 않습니다.
- 각 로그에는 시작/종료 마커, 단계별 진행, 모델 튜닝 결과, 산출물 경로, 총 소요 시간이 포함됩니다 — 문제 추적의 1차 자료.

---

19. 실행 이력·배포·롤백 `versioning`

역할

학습할 때마다 그 실행 전체를 `artifact_dir/runs/<실행 ID>/` 에 남기고, 운영 모델(`best.joblib`) 이 바뀔 때마다 배포 이력(`artifact_dir/deployments.jsonl`) 에 기록합니다. 그래서 새 모델이 운영에서 문제가 되면 **직전 운영 모델로 바로 되돌릴 수 있습니다.**

옵션

키	타입	기본값	설명
`auto_deploy`	bool	`true`	학습이 끝나면 best 를 바로 운영 모델로 배포. `false` 면 실행만 남기고, 배포는 `set_best` 로 직접
`keep_last_runs`	int	`5`	보관할 최근 실행 수 (1 이상). 현재 운영·직전 운영 실행은 이 수와 무관하게 보관

YAML 예시

```
versioning:
  auto_deploy: true
  keep_last_runs: 5
```

동작 메모

학습할 때 1. 후보 모델을 `runs/<실행 ID>/models/` 에 저장하고, `models/` 를 그 사본으로 통째로 교체합니다. 이전 실행의 후보가 섞이지 않습니다. 2. 리포트·test 예측도 실행 폴더에 사본을 남기고, `run_info.json` 에 설정·변수·데이터 정보· 모델별 점수·라이브러리 버전을 기록합니다. 3. `auto_deploy: true` 면 실행이 끝까지 완료된 뒤 best 를 운영 모델로 배포합니다. 4. 보관 개수를 넘는 오래된 실행을 지웁니다(운영·직전 운영 실행 제외).

운영 모델을 바꿀 때 (자동 배포 / `set_best` / `rollback` 공통) - 대상 모델을 실제로 열어 `scoring.input_path` 앞 100행으로 **시험 스코어링**을 합니다. 실패하면 운영 모델을 바꾸지 않습니다. - **모델**

이 쓰는 변수가 `scoring.input_path` 에 없으면 교체를 거부합니다. 그대로 두면 그 변수가 모든 행에서 학습 평균값(최빈값)으로 채워져 예측 품질이 조용히 떨어지기 때문입니다. 입력을 고치는 것이 우선이고, 알고도 진행하려면 `--allow-missing-features` 를 붙입니다. 변수 선택에서 빠진 변수만 없으면 WARNING 만 남깁니다(예측에 영향 없음). - 학습 후 자동 배포가 이 이유로 거부되면 실행·리포트는 저장된 채로 **에러로 끝나며** (cron 이 성공으로 오인하지 않도록) 운영 모델은 그대로입니다. 에러 메시지의 `set_best --run ...` 명령으로 확인 후 배포하세요. - 임시 파일에 쓴 뒤 한 번에 교체하므로, 교체 중 추론이 읽어도 깨진 파일을 보지 않습니다. - `deployments.jsonl` 에 시각·동작·실행·모델·점수·직전 운영 모델을 기록합니다.

동시 실행 잠금 - 학습(`train/*.py` , `AutoMLPipeline.run`)· `set_best` · `rollback` 은 실행 동안 `artifact_dir/.lock` 을 잡습니다. 다른 작업이 이미 잡고 있으면 기다리지 않고 **바로 에러**로 끝납니다(긴 학습을 마친 뒤에야 충돌을 알게 되는 일이 없도록 학습 시작 시점에 확인). - 작업이 비정상 종료되어 잠금이 남았으면, 같은 서버에서는 다음 작업이 자동으로 넘겨받습니다. 다른 서버(공유 디스크)에서 남긴 잠금은 판단할 수 없으므로, 그 작업이 정말 끝났는지 확인한 뒤 에러 메시지에 나온 `.lock` 파일을 지우세요. - 추론·SHAP 은 잠그지 않습니다(운영 모델 파일은 원자적으로 교체됨).

롤백

```
python -m auto_ml.cli.rollback --config <yaml>      # 직전 운영 모델로 (다시 실행하면 원래대로)
python -m auto_ml.cli.set_best --config <yaml> --run <실행 ID> --model <이름>   # 특정 실행으로
python -m auto_ml.cli.set_best --config <yaml> --list                           # 실행·배포 이력 보기
```

주의 - 실행마다 test 데이터·변수 구성이 다를 수 있습니다. 실행 간 점수를 그대로 비교하지 말고 `run_info.json` 을 함께 보세요. - 코드·라이브러리를 업데이트한 뒤에는 오래된 실행의 모델이 열리지 않을 수 있습니다. 이때 롤백은 시험 스코어링 단계에서 거부되고 운영 모델은 그대로 유지됩니다. - 예전 구조(`runs/` 없음)의 폴더는 다음 학습 때 기존 모델·리포트를 `runs/legacy_<원래 학습 시각>/` 으로 담아 두므로, 첫 롤백이 예전 운영 모델로 돌아갈 수 있습니다. - 같은 `artifact_dir` 의 학습 두 개는 잠금 때문에 동시에 돌지 않습니다. 단, 단계 스크립트 (`train/preprocess.py` → `train.py` → `postprocess.py`) 를 따로 실행하면서 두 학습의 단계를 섞어 실행하면(예: A 전처리 → B 전처리 → A 학습) 공유하는 `work/` 파일이 덮어써져 막을 수 없습니다. 학습은 하나씩 순서대로 실행하세요.

---

20. 운영 환경별 CPU 설정 (팟·컨테이너)

역할

학습·추론이 쓸 CPU 스레드 수를 정합니다. **운영 팟(컨테이너) 마다 CPU 할당이 다르므로, 배포 전에 팟별로 직접 값을 정해 설정 YAML 에 적는 것을 전제로 합니다.** 기본값(`null`) 은 기존 동작을 유지하기 위한 값일 뿐, 팟 환경에 맞춘 값이 아닙니다.

왜 필요한가

LightGBM·XGBoost·CatBoost 는 스레드 수를 지정하지 않으면 **프로세스마다 서버(노드) 의 모든 코어**만큼 스레드를 만듭니다. 실제로 쓸 수 있는 CPU 보다 스레드가 많아지면, 쉬는 스레드가 CPU 를 붙잡고 서로를 기다리면서 단순히 나눠 쓰는 것보다 훨씬 느려집니다. 측정 결과 (4코어, `examples/sample` 학습):

상황	기본 (`n_jobs: null`)	스레드를 나눈 경우
학습 1개 단독	약 7초	—
학습 2개 동시	**각 약 450초** (약 60배)	각 약 7~8초 (`training.n_jobs: 2`)
학습 중 스코어링 반복	**학습 약 440초**	학습 약 7.5초 (`training.n_jobs: 3` , `scoring.n_jobs: 1`)

환경 변수 `OMP_WAIT_POLICY=passive` 로도 동시 실행 때 느려짐이 크게 줄지만, 혼자 돌 때 학습이 약 2.2배 느려지므로 권하지 않습니다. `n_jobs` 를 정하는 것이 해결책입니다.

팟에서 특히 주의할 점

`n_jobs: null` 과 `-1` 은 "노드가 보여주는 전체 코어 수" 이지, 팟의 CPU limit 이 아닙니다. 쿠버네티스의 CPU limit 은 보통 CPU 사용 시간 할당량(cgroup quota) 으로 적용되는데, `nproc` · `os.cpu_count()` 와 각 라이브러리는 이것을 반영하지 않습니다. 예를 들어 32코어 노드에서 CPU limit 2 인 팟이면, `null` 일 때 **작업이 하나만 돌아도** 32개 스레드가 2 CPU 를 두고 경쟁해 위 표처럼 크게 느려질 수 있습니다. **limit 이 있는 팟에서는 항상 양의 정수로 명시하세요.** (정수 CPU 를 전용으로 할당하는 일부 설정에서는 코어 수에 반영되기도 하지만, 그에 기대지 말고 명시하는 것이 안전합니다.)

팟의 실제 CPU 확인 방법

방법	명령 / 위치	읽는 법
배포 매니페스트	`resources.limits.cpu`	`2` = 2 CPU, `1500m` = 1.5 CPU, `500m` = 0.5 CPU
팟 안 (cgroup v2)	`cat /sys/fs/cgroup/cpu.max`	`200000 100000` → 200000 ÷ 100000 = **2 CPU**. `max 100000` 이면 제한 없음
팟 안 (cgroup v1)	`cat /sys/fs/cgroup/cpu/cpu.cfs_quota_us /sys/fs/cgroup/cpu/cpu.cfs_period_us`	첫 값 ÷ 둘째 값 = CPU 수. 첫 값이 `-1` 이면 제한 없음
⚠️ 쓰면 안 되는 값	`nproc` , `os.cpu_count()`	노드 코어 수 — limit 이 반영되지 않음

limit 없이 request 만 있는 팟은 같은 노드의 다른 팟과 CPU 를 나눠 쓰므로, request 값을 기준으로 정하는 것을 권합니다.

값 정하는 규칙

팟의 CPU 를 `L` 이라고 할 때:

팟 구성	`training.n_jobs`	`scoring.n_jobs`
학습만 도는 팟	`L` 의 정수 부분 (최소 1)	—
추론(스코어링·SHAP) 만 도는 팟	—	`L` 의 정수 부분 (최소 1)
같은 팟에서 학습과 스코어링 cron 이 겹침	예: `L - 1`	예: `1` — **두 값의 합 ≤ L**
limit 1 미만 (`500m` 등)	`1`	`1`
limit 없는 전용 서버	`null` 가능. 단 같은 서버에서 동시에 도는 작업들의 스레드 합 ≤ 코어 수	동일

```
# 예: CPU limit 4 팟에서 학습과 스코어링 cron 이 겹치는 경우
training:
  n_jobs: 3
scoring:
  n_jobs: 1
```

설정 위치와 우선순위

키	적용 범위	`null` 일 때
`training.n_jobs`	튜닝·CV·최종 학습·변수 선택(LightGBM)	라이브러리 기본 = 노드 전체 코어
`scoring.n_jobs`	스코어링(`inference/*.py`)·SHAP· `run_scoring`	**학습 때 값이 모델에 저장돼 그대로 쓰임** (학습도 `null` 이었으면 노드 전체 코어)

- 음수는 sklearn 규칙입니다 (`-1` = 전체, `-2` = 하나 남김). 위 이유로 팟에서는 쓰지 마세요.
- 모델 `fixed_params` 에 스레드 인자(`num_threads` · `nthread` · `thread_count` 등) 를 직접 쓰면 그 값이 우선합니다.
- `n_jobs` 는 **속도만 바꾸고 예측 결과는 바꾸지 않습니다** (기존에 학습된 모델 파일에도 적용됨).
- 학습 팟과 추론 팟의 CPU 가 다르면, 추론 쪽 YAML 에 `scoring.n_jobs` 를 따로 적으세요. 비워 두면 학습 팟 기준의 값이 추론 팟에서도 쓰입니다.

배포 전 체크리스트

1. 학습·추론이 돌 팟의 CPU limit(또는 request) 을 확인한다 (위 표의 방법).
2. 설정 YAML 에 `training.n_jobs`, `scoring.n_jobs` 를 양의 정수로 적는다 (같은 팟에서 겹치면 합 ≤ limit).
3. 그 팟에서 짧은 학습을 한 번 돌려 소요 시간을 확인한다. 예상보다 몇 배 이상 느리면 같은 팟에서 동시에 도는 다른 작업이 있는지, 값이 limit 보다 크지 않은지 다시 본다.

부록

A. 실행 명령어 모음

모두 저장소 루트에서 실행합니다. 단계 스크립트는 `--config` 외에 `--work-dir` (선택) 을 받습니다.

명령	용도
`python train/preprocess.py --config <yaml>`	학습 1/3 전처리
`python train/train.py --config <yaml>`	학습 2/3 모델 학습
`python train/postprocess.py --config <yaml>`	학습 3/3 리포트·artifact 저장
`python train/run_all.py --config <yaml>`	학습 전체
`python inference/preprocess.py --config <yaml>`	추론 1/3 전처리
`python inference/predict.py --config <yaml>`	추론 2/3 예측
`python inference/postprocess.py --config <yaml>`	추론 3/3 결과 저장
`python inference/run_all.py --config <yaml>`	추론 전체
`python -m auto_ml.explain.runner --config <yaml>`	SHAP 해석
`python -m auto_ml.cli.set_best --config <yaml> --model <name>`	재학습 없이 운영 모델 교체 (최신 실행 후보)
`python -m auto_ml.cli.set_best --config <yaml> --run <id> --model <name>`	이전 실행의 모델로 교체
`python -m auto_ml.cli.set_best --config <yaml> --list`	실행 목록·배포 이력 보기
`python -m auto_ml.cli.rollback --config <yaml>`	직전 운영 모델로 되돌리기 (다시 실행하면 원래대로)

`set_best` · `rollback` 은 `--allow-missing-features` 를 받습니다 — 모델이 쓰는 변수가 스코어링 입력에 없어도 교체합니다(기본은 거부, § 19).

best 교체 예시:

```
# xgb 후보를 best 로 승격
python -m auto_ml.cli.set_best --config configs/my_config.yaml --model xgb

# 앙상블 후보를 best 로 승격
python -m auto_ml.cli.set_best --config configs/my_config.yaml --model ensemble
```

`best.joblib` 의 메타데이터는 배포한 후보 파일의 메타데이터와 같고(`run_id` 포함), `extra` 에 배포 정보가 추가됩니다. - `best_selection` : `auto_by_metric` (자동 선정) / `user_override` (`training.best_model` 지정) / `promoted` (set_best) / `rollback` - `deployed_at` - `promoted_at` / `promoted_from` : set_best·rollback 인 경우만. `promoted_from` 은 이전 운영 모델 이름

롤백·실행 이력은 § 19 참고.

B. 산출물 안내

B.1 `best.joblib` (학습 산출물)

전처리기 + 모델 + 메타데이터가 하나의 파일로 묶인 cloudpickle+gzip 번들입니다. 스코어링/ SHAP 시 이 파일 하나만 있으면 됩니다.

전체 구조:

```
<artifact_dir>/
├── best.joblib                    ← 운영 모델 (scoring/explain 대상)
├── deployments.jsonl              ← 배포 이력
├── models/                        ← 최신 실행 후보 사본 (set_best 기본 대상)
│   ├── lgbm.joblib  xgb.joblib  catboost.joblib
│   ├── elasticnet.joblib          ← enabled 시
│   └── ensemble.joblib            ← 앙상블이 생성된 경우
└── runs/<실행 ID>/               ← 실행 스냅샷 (최근 5개 + 운영·직전 운영 실행 보관)
    ├── models/*.joblib
    ├── reports/  predictions/
    └── run_info.json
```

`best.joblib` 은 후보 파일 중 하나의 복사본입니다. 각 후보 파일은 독립 번들이라 그 자체로 스코어링 / SHAP 해석에 사용할 수 있습니다. 실행 이력·롤백은 § 19 참고.

B.2 HTML / PDF 리포트

`reporting.output_dir/report.html` 을 브라우저로 열면 8개 섹션으로 구성된 리포트를 볼 수 있습니다.

섹션	내용
§ 1 모델 비교 (holdout 기준)	전 모델 Test 점수 비교표 + best 하이라이트
§ 2 CV (OOF) 비교	Out-Of-Fold 평가 비교표
§ 3 오버핏 점검 (Train vs Holdout)	Train·Test 점수 Δ 표 — 과적합 진단
§ 4 곡선 비교	ROC Curve + Precision-Recall Curve
§ 5 Best 모델 상세	Feature Importance (상위 N), 10-분위 분석, Score Distribution, Confusion Matrix
§ 6 학습 설정 요약	사용된 YAML 설정 핵심 요약
§ 7 모델 파라미터	각 모델의 최종 파라미터 (Optuna best params 포함)
§ 8 변수 선택	(활성 시) Stability Selection 빈도 막대 + 채택/제외 라벨

PDF 도 동일 내용입니다.

부속 파일 - `feature_importance.csv` — 전체 변수 중요도 (HTML 은 `top_importance_features` 만 표시하지만 CSV 는 항상 전체). - `feature_selection.csv` — (변수 선택 활성 시) 변수별 선택 빈도. 모형 무관이라 메인 리포트에만 1개.

비-best 모형 sub-report

best 가 아닌 모형마다 `reporting.output_dir/sub/<model>/` 에 별도 리포트가 자동 생성됩니다.

```
reports/sub/lgbm/report.html, report.pdf, feature_importance.csv
reports/sub/xgb/...
reports/sub/ensemble/...
```

각 sub-report 는 해당 모형 vs best 점수 비교, feature importance, 10-분위 분석, score distribution, confusion matrix, 변수 선택 결과를 포함합니다. 이번 실행에 없는 모형의 sub-report 폴더는 지워집니다(이전 실행의 리포트는 `runs/<실행 ID>/reports/` 에 사본이 남아 있습니다).

B.3 `scores.parquet` (스코어링 산출물)

```
user_id, score, prediction
1001,    0.87,  1
1002,    0.12,  0
```

- `score` — 0~1 사이 확률값
- `prediction` — `score >= scoring.threshold` 면 1, 아니면 0

B.4 `test_predictions.parquet` (학습 부산물)

학습 시 best 모델의 holdout 예측을 함께 export 합니다: `<artifact_dir>/../predictions/test_predictions.parquet`, 스키마 `<id_columns> + score + prediction + <target_column>`. 외부 BI / 추가 진단용 (overfit 검증, 임계값 튜닝 등).

B.5 `shap.parquet` (SHAP 산출물)

`<id_columns> + shap_<feature>... + base_value + score` wide 포맷. 확률 도메인이며 `base_value + sum(shap_*) ≈ score` 가법성이 보장됩니다. 자세한 스키마는 § 17 SHAP 해석 참고.

B.6 로그 파일

`logging.log_dir` (기본 `artifacts/logs/`) 에 실행 단위로 로그가 생성됩니다.

```
train_preprocess_YYYYMMDD_HHMMSS.log       ← train/preprocess.py
train_train_YYYYMMDD_HHMMSS.log            ← train/train.py
train_postprocess_YYYYMMDD_HHMMSS.log      ← train/postprocess.py
train_YYYYMMDD_HHMMSS.log                  ← train/run_all.py,
AutoMLPipeline.run()
inference_preprocess_YYYYMMDD_HHMMSS.log   ← inference/preprocess.py
inference_predict_YYYYMMDD_HHMMSS.log      ← inference/predict.py
inference_postprocess_YYYYMMDD_HHMMSS.log  ← inference/postprocess.py
inference_YYYYMMDD_HHMMSS.log              ← inference/run_all.py
score_YYYYMMDD_HHMMSS.log                  ← run_scoring() (Python API)
explain_YYYYMMDD_HHMMSS.log                ← python -m auto_ml.explain.runner
```

단계별 진행, 모델 튜닝 결과, 산출물 경로, 총 소요시간이 기록됩니다. 문제 발생 시 가장 먼저 보는 곳입니다.

B.7 `work/` (단계 간 중간 산출물)

`<artifact_dir 의 상위>/work/{train,inference}/*.pkl.gz` — 단계 스크립트가 다음 단계에 넘기는 파일입니다 (cloudpickle + gzip). 내용과 재실행 규칙은 § 5.3 참고. 운영 스코어링에는 필요 없으며, 디스크가 부족하면 지워도 됩니다 (다음 실행 때 다시 생성).

C. 트러블슈팅 FAQ

Q1. **`ModuleNotFoundError`** **(예:** **`No module named 'lightgbm'`** **)** 가상환경 활성화 여부 확인. 프롬프트 앞에 `(.venv)` 가 보여야 합니다. 안 보이면 `source .venv/bin/activate` 다시 실행하고 `pip install -r requirements.txt` 를 확인. `python -m auto_ml...` 형태 명령에서 `No module named 'auto_ml'` 이 나오면 저장소 루트에서 실행했는지 확인.

Q2. **`ValueError: target column must contain only 0 and 1`** 타깃 컬럼에 0/1 외 값(NaN, 2, "yes" 등)이 섞임. 데이터 정리 후 재시도.

Q3. **`ValueError: column X is all NaN`** 어떤 수치형 컬럼이 학습 데이터에서 전부 결측. 다음 중 하나로 해결: - `features.csv` 에서 해당 컬럼 `used: false` - `preprocessing.numeric_null_strategy: constant` 로 전환 - 데이터 자체 정리

Q4. PDF 리포트가 안 만들어져요 WeasyPrint 시스템 의존성 누락. HTML 은 정상 생성됩니다. PDF 가 필요하면 §3 의 시스템 패키지 설치, 아니면 `reporting.generate_pdf: false` 로 끄세요.

Q5. 학습이 너무 오래 걸립니다 다음을 줄이세요:

```
tuning:
  n_trials: 10           # 30 → 10
  cv_folds: 2            # 3 → 2
training:
  cv_folds: 3            # 5 → 3
```

또는 `tuning.timeout: 1800` (30분) 으로 wall-clock 상한 지정.

Q6. 모델 한 개만 쓰고 싶어요 `models.<name>.enabled: false` 로 나머지를 끄세요. 최소 1개는 켜져 있어야 합니다.

Q7. 임계값을 0.5 가 아닌 값으로 `scoring.threshold: 0.3` 처럼 변경 (0.3 이상이면 1로 분류).

Q8. 스코어링 입력에 컬럼 하나가 없어요 에러 없이 진행됩니다. 빠진 feature 컬럼은 학습 시 기본값(수치형 median, 범주형 최빈값)으로 채워지고 `Input is missing N feature column(s); filled with training defaults: [...]` WARNING 이 남습니다. 해당 변수 정보 없이 예측한 셈이므로, 일시 누락이 아니라면 입력 데이터를 고치세요.

Q9. 오버핏 Δ 가 너무 큽니다 §11 최종 모델 학습 전략의 `final_fit_strategy: iteration_capping` 또는 `cv_bagging` 으로 전환을 검토하세요. 테스트셋이 학습 신호로 노출되는 편향을 제거합니다.

Q10. ElasticNet 만 Focal Loss 가 안 됩니다 지원하지 않는 조합입니다. `models.elasticnet.loss: logloss` (기본값) 로 두세요.

Q11. **`FileNotFoundError: Stage output not found: ... Run the previous stage first.`** 앞 단계 스크립트를 실행하지 않았거나, 앞 단계와 다른 `--work-dir` / 다른 config(`artifact_dir`) 를 줬을 때 납니다. 앞 단계를 먼저 실행하고, 모든 단계에 같은 `--config` · `--work-dir` 를 주세요.

Q12. `FileNotFoundError: Artifact not found at .../best.joblib` 추론 전에 학습(`train/postprocess.py` 까지) 이 끝나야 합니다. 추론 config 의 `artifact_dir` 이 학습 때와 같은지도 확인하세요.

Q13. 설정을 바꿨는데 결과가 그대로예요 뒷 단계만 다시 실행하면 앞 단계의 옛 결과를 그대로 씁니다. § 5.3 의 표에서 바꾼 설정에 해당하는 단계부터 다시 실행하세요.

Q14. 로그에 `unseen categories in '<컬럼>'` 경고가 나와요 추론 입력에 학습 때 없던 범주 값이 그 비율만큼 있다는 뜻입니다. 경고에 예시 값이 함께 나옵니다. 새 코드가 생긴 것이라면 재학습을 검토하고, 표기만 다른 것(대소문자·공백·오타) 이라면 입력 데이터를 학습 때와 같은 표기로 맞추세요.

Q15. 새로 학습한 모델이 운영에서 이상해요. 이전 모델로 되돌리려면?
`python -m auto_ml.cli.rollback --config <yaml>` 로 직전 운영 모델로 돌아갑니다. 한 번 더 실행하면 다시 새 모델로 돌아옵니다. 더 이전 실행으로 가려면 `set_best --list` 로 실행 ID 를 확인한 뒤 `set_best --run <실행 ID> --model <이름>` 을 쓰세요. 대상 모델이 열리지 않거나 시험 스코어링에 실패하면 롤백이 거부되고 운영 모델은 그대로입니다 (§ 19).

Q16. `runs/` 폴더가 계속 커지지 않나요? `versioning.keep_last_runs` (기본 5) 를 넘는 오래된 실행은 학습할 때 자동으로 지워집니다. 단, 현재 운영 실행과 직전 운영 실행은 오래돼도 남깁니다. 실행 하나의 크기는 후보 모델 + 리포트(PDF 포함) + test 예측 정도입니다.

Q17. 학습·스코어링이 예상보다 수십 배 느려요 (특히 팟·컨테이너에서) 스레드 수가 실제로 쓸 수 있는 CPU 보다 많은 경우입니다. 같은 서버에서 학습과 스코어링(또는 학습 두 개) 이 동시에 돌거나, CPU limit 이 있는 팟에서 `n_jobs` 를 비워 둔 경우(노드 전체 코어만큼 스레드를 만듦) 에 생깁니다. 팟의 CPU 를 확인해 `training.n_jobs` · `scoring.n_jobs` 를 양의 정수로 적으세요. 확인 방법·값 정하는 규칙·측정 수치는 § 20 참고.

D. 폐쇄망 이관

다음을 옮기면 됩니다.

1. **패키지 wheel 묶음** — 외부망에서 미리 만들어 둡니다. `bash pip wheel -r requirements.txt -w wheelhouse/`
2. **본 저장소 자체** — `git clone` 한 폴더 그대로 (`auto_ml/` , `train/` , `inference/` 가 필요합니다. 설치는 하지 않습니다).
3. **학습 산출물** — `artifacts/models/best.joblib` (`work/` 폴더는 옮길 필요 없음. 폐쇄망에서 롤백도 하려면 `runs/` 와 `deployments.jsonl` 도 함께 옮기세요)
4. **스코어링용 설정 YAML 과 그 YAML 의 `features_csv` 가 가리키는 CSV** — 설정을 읽을 때 CSV 도 함께 읽으므로, CSV 가 없으면 `FileNotFoundError` 로 실패합니다.

폐쇄망에서 설치:

```
pip install --no-index --find-links=wheelhouse -r requirements.txt
python inference/run_all.py --config configs/my_config.yaml
```

PDF 리포트가 필요하면 WeasyPrint 시스템 의존성(libpango 등)도 함께 배포하세요.

E. 용어 사전

용어	설명
이진 분류	0/1 (참/거짓) 두 가지 중 하나를 맞히는 문제
타깃 (target)	예측 대상 컬럼 (0 또는 1)
피처 (feature)	예측의 근거가 되는 입력 변수
전처리	결측 채우기, 이상치 자르기, 스케일링 등 학습 전 데이터 정리
스케일링	변수마다 다른 단위를 비슷한 크기로 맞춤
이상치	분포에서 극단적으로 동떨어진 값
Skew (왜도)	분포가 한쪽으로 치우친 정도
Stability Selection	부분표본을 여러 번 뽑아 자주 선택되는 변수만 추리는 절차 (Meinshausen & Bühlmann, 2010)
Optuna / TPE	베이지안 하이퍼파라미터 탐색 라이브러리·알고리즘
하이퍼파라미터	학습 전 사람이 정하는 설정값 (학습률, 트리 깊이 등)
KFold / OOF	데이터 K 등분 후 돌아가며 평가. Out-Of-Fold 예측으로 안정적 점수 산출
Early stopping	검증셋 점수가 일정 라운드 개선되지 않으면 학습 조기 종료
ROC-AUC	분류 성능 지표 (0.5=찍기, 1.0=완벽). 본 라이브러리 기본 지표
PR-AUC	Precision-Recall 곡선 아래 면적. positive 비율이 매우 낮을 때 권장
KS statistic	Kolmogorov-Smirnov 통계. 신용평점 도메인 표준 지표
Focal Loss	어려운 샘플에 학습 신호를 집중시키는 손실 (Lin et al. 2017). 불균형 데이터 대응
SHAP	건별·변수별 예측 기여도. Shapley value 기반
Artifact	학습 결과물(모델 + 전처리기 + 메타데이터) 단일 번들 파일
단계 (stage)	학습·추론을 나눈 실행 단위. 학습 = 전처리 → 학습 → 후처리, 추론 = 전처리 → 예측 → 후처리
work_dir	단계 스크립트가 다음 단계에 넘기는 중간 산출물(`*.pkl.gz`) 을 두는 폴더
Parquet	컬럼 기반 효율적 데이터 파일 형식 (CSV 보다 빠르고 작음)

F. 변경 이력과 업그레이드 안내

F.1 변경 요약

이전에 반영된 변경 - **실행 방식:** 패키지 설치 없이 `train/` · `inference/` 의 `.py` 스크립트로 실행합니다(전처리 → 학습/예측 → 후처리). - **범주형 처리:** 범주형 값을 문자열로 정규화(`1` / `1.0` / `"1"` → `"1"`)하고, 학습 때 없던 범주 비율을 로그로 남깁니다. XGBoost 는 자체 범주형 처리(`enable_categorical`)를 씁니다.

이번 변경

묶음	내용
문서·예시	문서의 사실관계 오류 수정. 예시 설정(`configs/example.yaml` , titanic) 에 ElasticNet 을 켜서 기본 앙상블이 실제로 만들어지도록 함. `examples/make_dummy_data.py` 가 학습·테스트·추론 데이터를 같은 분포에서 만들도록 수정
정확도	XGBoost SHAP 이 조기 종료 이후 트리까지 포함하던 문제 수정. 앙상블 SHAP 을 확률 기준으로 계산해 `shap.parquet` 의 score 가 스코어링 점수와 정확히 일치. `best_iteration` 을 모든 모델에서 "트리 개수" 로 통일(조기 종료 없으면 비움). 서로 다른 모델이 섞인 앙상블의 변수 중요도를 비율로 맞춰 합산. 아주 큰 정수 범주 코드의 정밀도 손실 수정
안정성	PDF 생성이 실패해도 모델은 저장(PDF 만 건너뜀). 타깃에 결측이 있으면 학습 시작 시 명확한 에러. 모델 파일을 원자적으로 저장. `best.joblib` 메타데이터를 후보 파일과 통일
실행 이력·롤백	학습마다 `runs/<실행 ID>/` 에 실행 전체 보관, `deployments.jsonl` 배포 이력, `rollback` · `set_best --run` · `set_best --list` , 보존 개수 설정, 예전 폴더 자동 이전 (§ 19)
배포 안전장치	모델이 쓰는 변수가 스코어링 입력에 없으면 배포 거부(`--allow-missing-features` 로만 허용). 학습·배포 동시 실행 잠금(`.lock`)
CPU	`training.n_jobs` , `scoring.n_jobs` 로 스레드 수 지정 (§ 20)

F.2 업그레이드할 때 확인할 것

적용 시점

언제 적용되나	해당 변경
업그레이드 즉시 (기존 `best.joblib` 그대로)	XGBoost·앙상블 SHAP 정확도, `scoring.n_jobs` , 배포 안전장치·잠금, 원자적 저장
다음 학습부터	`best_iteration` 표시, 앙상블 변수 중요도, 메타데이터 통일, PDF 실패 처리, 결측 타깃 검사, 실행 이력(`runs/`), 범주형 정규화, `training.n_jobs`

운영 스크립트(cron 등) 에 영향을 줄 수 있는 동작 변화 - 학습 후 자동 배포가 **변수 누락으로 거부되면 학습이 에러(종료 코드 0 아님) 로 끝납니다.** 실행은 저장되고 운영 모델은 그대로입니다. - 같은 `artifact_dir` 에서 다른 학습· `set_best` · `rollback` 이 진행 중이면 **바로 에러로 끝납니다.** - PDF 생성 실패는 더 이상 학습을 중단시키지 않습니다(WARNING 후 정상 종료). - `models/` 폴더는 최신 실행의 사본으로 **통째로 교체**됩니다. 이전 실행의 sub-report 폴더(`reports/sub/<모델>/`) 는 지워지고, 실행 폴더에 사본이 남습니다. - 예전 구조의 `artifact_dir` 은 첫 학습 때 `runs/legacy_<원래 학습 시각>/` 으로 담깁니다. - 새 파일·폴더가 생깁니다: `artifact_dir/runs/` , `deployments.jsonl` , 작업 중의 `.lock` (저장소 예시 산출물에서는 git 제외 처리됨). - 리포트 표시 변화: XGBoost·CatBoost 의 `best_iter` 가 이전보다 1 크게 표시되고(이제 정확한 값), 서로 다른 모델이 섞인 앙상블의 `feature_importance.csv` 의 `importance_gain` 컬럼은 비율(합 1) 값입니다.

새 설정 키 — 모두 기본값이면 기존 동작과 같습니다.

키	기본값	참고
`versioning.auto_deploy`	`true`	§ 19
`versioning.keep_last_runs`	`5`	§ 19
`training.n_jobs`	`null`	**팟에서는 반드시 명시** — § 20
`scoring.n_jobs`	`null`	**팟에서는 반드시 명시** — § 20

배포 전 권장 순서 1. 팟별 CPU 를 확인해 `training.n_jobs` · `scoring.n_jobs` 를 적습니다 (§ 20 체크리스트). 2. 스코어링 입력(`scoring.input_path`) 이 학습 변수를 모두 갖추고 있는지 확인합니다(없으면 자동 배포가 거부됨). 3. 학습을 한 번 돌려 `python -m auto_ml.cli.set_best --config <yaml> --list` 로 실행·배포 이력을 확인합니다. 4. cron 등 운영 스크립트가 학습 종료 코드를 확인하도록 되어 있는지 봅니다.

F.3 알려진 제약

- `n_jobs: null` · `-1` 은 팟의 CPU limit 을 자동으로 읽지 않습니다(노드 코어 수 기준). 팟에서는 값을 명시하세요.
- 단계 스크립트를 따로 실행하면서 두 학습의 단계를 섞으면 공유 `work/` 가 덮어써져 막을 수 없습니다.
- 다른 서버(공유 디스크) 에서 비정상 종료로 남은 잠금은 자동으로 넘겨받지 않습니다. 작업이 끝났는지 확인한 뒤 `.lock` 파일을 지우세요.
- 코드·라이브러리를 업데이트한 뒤에는 오래된 실행의 모델이 열리지 않을 수 있습니다. 이 경우 롤백은 시험 스코어링 단계에서 거부되고 운영 모델은 유지됩니다.

문제가 풀리지 않으면 `artifacts/logs/` 의 최신 로그 파일과 함께 담당자에게 공유해 주세요.